arXiv:2603.28052v1 [cs.AI] 30 Mar 2026

Meta-Harness: End-to-End Optimization of Model Harnesses

Yoonho Lee
Stanford

Roshen Nair
Stanford

Qizheng Zhang
Stanford

Kangwook Lee
KRAFTON

Omar Khattab
MIT

Chelsea Finn
Stanford

Project page w/ interactive demo: `https://yoonholee.com/meta-harness/`
Optimized harness: `https://github.com/stanford-iris-lab/meta-harness-tbench2-artifact`

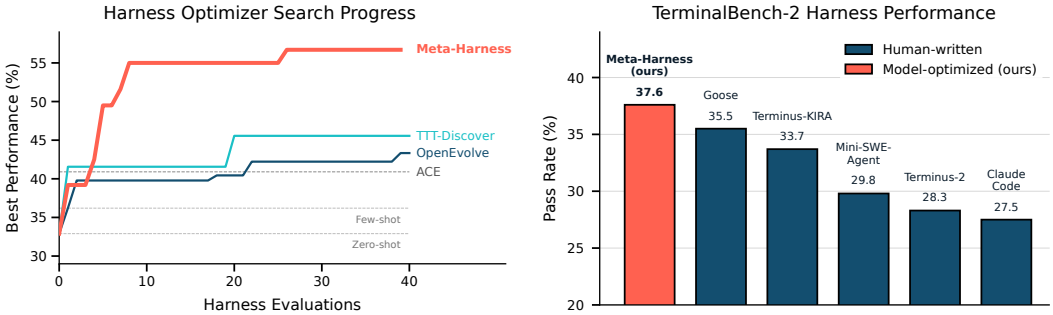

Figure 1: **(Left)** On text classification, Meta-Harness outperforms the best prior hand-designed harnesses (ACE) and existing text optimizers (TTT-Discover, OpenEvolve), matching the next-best method's final accuracy after just 4 evaluations. **(Right)** On TerminalBench-2, Meta-Harness outperforms all reported `Claude Haiku 4.5` harnesses.

Abstract

The performance of large language model (LLM) systems depends not only on model weights, but also on their *harness*: the code that determines what information to store, retrieve, and present to the model. Yet harnesses are still designed largely by hand, and existing text optimizers are poorly matched to this setting because they compress feedback too aggressively: they are memoryless, condition only on scalar scores, or restrict feedback to short templates or summaries. We introduce **Meta-Harness**, an outer-loop system that searches over harness code for LLM applications. It uses an agentic proposer that accesses the source code, scores, and execution traces of all prior candidates through a filesystem. On online text classification, Meta-Harness improves over a state-of-the-art context management system by 7.7 points while using 4× fewer context tokens. On retrieval-augmented math reasoning, a single discovered harness improves accuracy on 200 IMO-level problems by 4.7 points on average across five held-out models. On agentic coding, discovered harnesses surpass the best hand-engineered baselines on TerminalBench-2. Together, these results show that richer access to prior experience can enable automated harness engineering.

1 Introduction

Changing the harness around a fixed large language model (LLM) can produce a 6× performance gap on the same benchmark [47]. The *harness*—the code that determines what to store, retrieve, and show to the model—often matters as much as the model itself. This sensitivity has led to growing interest in **harness engineering**, the practice of refining the code around an LLM to improve the overall system's performance [36; 21; 10; 9]. But despite its importance, harness engineering remains largely manual: practitioners inspect failures,

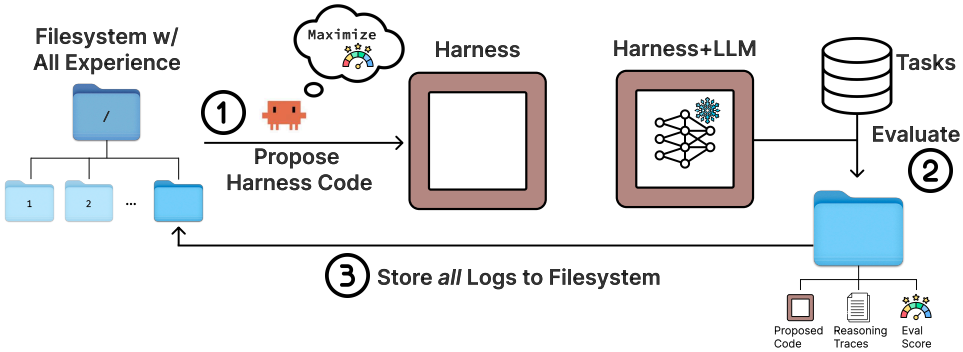

Figure 2: **Meta-Harness search loop. (1)** An agent reads a filesystem containing all prior candidates' source code, execution traces, and scores, and proposes a new harness. **(2)** We evaluate the proposed harness on evaluation tasks. **(3)** All logs (proposed code, reasoning traces, evaluation scores) are stored in the filesystem in a new directory, and the loop repeats.

Method	History	Log content	MTok/iter
OPRO [51]	Window	past (solution, score) pairs	0.002
TextGrad [53]	Last	textual feedback on current artifact	0.015
AlphaEvolve [35]	Window	program database + eval. scores	0.022
GEPA [1]	Summary	reflective feedback from rollout traces	0.008
Feedback Descent [26]	Summary	comparison + textual feedback	0.012
TTT-Discover [54]	Window	prev. solution fragment	0.026
`Meta-Harness`	**Full**	***all*** logs and scores	10.0

Table 1: **Comparison of text optimization methods and their settings.** Each row represents a method collapsed across tasks. `Mtok/iter` is our best estimate of the full context generated from one evaluation of a text artifact in the *largest setting considered in each paper*. This paper considers settings that yield orders-of-magnitude more context per artifact evaluation.

adjust heuristics, and iterate on a small number of designs. In this paper, we ask whether this process itself can be automated.

A natural starting point is recent work on text optimization, since harness engineering also involves iteratively improving text and code artifacts using feedback from prior attempts [38; 39; 35; 26; 1]. However, these methods are poorly matched to harness engineering because they typically operate with short-horizon or heavily compressed feedback: some condition only on the current candidate [31; 51; 53], others rely primarily on scalar scores [35; 12], and others restrict feedback to short templates or LLM-generated summaries [1; 26]. This is a pragmatic scalability choice, not evidence that longer-range dependencies are uninformative. Harnesses act over long horizons: a single choice about what to store, when to retrieve it, or how to present it can affect behavior many reasoning steps later. Compressed feedback often removes the information needed to trace downstream failures to earlier harness decisions. Across the tasks studied by several representative text optimizers, the available context per optimization step ranges from only 100 to 30,000 tokens (Table 1), far below the diagnostic footprint of harness search. More broadly, work on retrieval and memory-augmented language models suggests that useful context should often be accessed adaptively rather than monolithically packed into a single prompt [28; 48; 37; 56].

We address this limitation with **Meta-Harness**, an agentic harness for optimizing harnesses via end-to-end search (Figure 2). Its proposer is a coding agent, i.e., a language-model-based system that can invoke developer tools and modify code. The choice of coding agent (rather than raw LLM) matters because the amount of experience quickly exceeds context limits, so the proposer must decide *what* to inspect and validate edits through direct interaction with the codebase. Its key design choice is to expose **full history** through a *filesystem*, enabling selective diagnosis of raw prior code and execution traces rather than optimization from compressed per-candidate summaries. For every previous candidate harness, the filesystem stores the source code, evaluation scores, and execution traces, which the proposer retrieves via standard operations such as `grep` and `cat` rather than ingesting them as a single prompt. In practice, the proposer reads a median of **82 files per iteration** in our most demanding setting, referencing over 20 prior candidates per step (Appendix A). In the settings we

study, a single evaluation can produce up to 10,000,000 tokens of diagnostic information, roughly three orders of magnitude beyond the largest feedback budgets used in prior text optimization settings (Table 1).

We evaluate Meta-Harness on online text classification, mathematical reasoning, and agentic coding. On online text classification, harnesses discovered by Meta-Harness improve over Agentic Context Engineering (ACE, Zhang et al. [59]) by **7.7 points** while using 4× fewer context tokens, and match the next-best text optimizer's final performance after 60 proposals with only four (Figure 1). On retrieval-augmented math reasoning, a single discovered harness improves accuracy on 200 IMO-level problems by **4.7 points** on average across five held-out models. On TerminalBench-2, the discovered harness **surpasses Terminus-KIRA and ranks #1 among all Haiku 4.5 agents**.

2 Related Work

At a high level, Meta-Harness brings ideas from the broader literature on credit assignment and meta-learning [40; 46; 3; 17; 44; 2] in a new regime enabled by recent advances in coding agents. Rather than updating model weights, the system assigns credit at the harness level: it uses experience from past rollouts to deliberately reason about which steps and components are responsible for failures, then rewrites the external code that governs future behavior. More specifically, the method lies at the intersection of several recent research threads; it is most directly related to work on adaptive access to external context, executable code search, and text optimization.

External memory and adaptive access. Several prior works note the benefits of treating large knowledge sources or long inputs as external resources that a language model accesses adaptively, rather than consuming them in a single pass. Specifically, retrieval-augmented generation [28], interleaved retrieval and reasoning [48], memory-based agents [37], or recursive language models [56] are mechanisms for adaptive access to external context. Meta-Harness uses a similar access pattern, but in the more demanding setting of harness engineering, where the proposer selectively inspects a large external history of code, scores, and execution traces to improve context-management procedures themselves.

Executable code search. Recent methods search over executable code for functions, workflows, or agent designs. Early work proposes using large models as mutation and crossover operators in evolutionary program search [27]. Later methods evolve designated functions within fixed program scaffolds [39], use meta-agents to program new agents from prior discoveries [20], or search over workflow graphs for agentic systems [58]. Another line of work searches over memory designs for continual-learning agents, where memory persists across task streams [57; 50]. In contrast, Meta-Harness searches over domain-specific harnesses, including prompt construction, retrieval, and state update strategies that reset between tasks. Its outer loop is deliberately minimal: instead of relying on a fixed scaffold, an archive of prior discoveries, or a persistent memory mechanism, it gives the proposer unrestricted filesystem access to prior experience. This lets the agent decide what information to inspect and enables search over full harness implementations rather than a predefined space of context-management procedures.

Text optimization methods. Meta-Harness is also closely related to methods such as ProTeGi, TextGrad, OPRO, GEPA, AlphaEvolve/OpenEvolve, and Feedback Descent, which iteratively improve prompts or other text artifacts using feedback from prior attempts [38; 31; 53; 51; 1; 35; 43; 26]. However, these methods are less well suited to harness engineering, where optimization targets a complete executable procedure, and the relevant environmental feedback is distributed across code, scores, and execution traces in a way that is hard to summarize up front. Rather than reacting only to aggregate scores or summaries, the proposer in Meta-Harness can reason over failed examples and their execution traces to propose targeted edits. See Table 1 for a comparison of problem scale considered in those papers and ours, and Figures 1 and 4 for a direct comparison with OpenEvolve, GEPA, and TTT-Discover in our problem setting.

3 Meta-Harness: A Harness for Optimizing Harnesses

This section describes Meta-Harness, our outer-loop procedure for searching over task-specific harnesses. Meta-Harness is built on the idea that harness optimization benefits from allowing a proposer to selectively inspect prior code and execution traces via filesystem access, rather than optimizing from lossy summaries or an additional hand-designed search structure. At a high level, it repeatedly proposes, evaluates, and logs new harnesses.

Meta-Harness is itself a harness in the broad sense (hence the name), since it determines what information the proposer model sees during search. Unless otherwise noted, we use *harness* to refer to the task-specific programs being optimized.

Objective. A harness is a stateful program that wraps a language model and determines what context the model sees at each step. The goal is simple: find the harness that makes the underlying model perform best on the target task distribution. Formally, let M denote a fixed language model and $\mathcal{X}$ a task distribution. For a harness H and task instance $x \sim \mathcal{X}$, we execute a rollout trajectory $\tau \sim p_M(H, x)$. The harness constructs prompts for M, the model responds, and the harness updates its state after each interaction. A task-specific reward function $r(\tau, x)$ scores the trajectory. The objective of harness optimization is to **find the harness that maximizes the expected final reward**:

$$H^* = \arg\max_H \mathbb{E}_{x\sim\mathcal{X},\tau\sim p_M(H,x)}\, r(\tau, x),$$

When multiple objectives are relevant (e.g., accuracy and context cost), we evaluate candidates under Pareto dominance and report the resulting frontier. In practice, this search has traditionally been carried out by human engineers and researchers, who iteratively refine prompts, context-management rules, and tool-use logic by hand.

Meta-Harness search loop. Meta-Harness uses a single coding-agent proposer with access to a growing filesystem $\mathcal{D}$ that serves as its feedback channel[1]. Here, a *coding agent* is a language-model-based system that can invoke developer tools and modify code. Unlike prior systems that externalize the improvement logic in a hand-designed search loop, Meta-Harness delegates diagnosis and proposal to the coding agent itself: it decides which prior artifacts to inspect, which failure modes to address, and whether to make a local edit or a more substantial rewrite. Equivalently, the proposer is not a raw next-token model operating on a fixed prompt assembled by the outer loop; it is an agent that retrieves information, navigates prior artifacts, and edits code as part of the search itself. Each evaluated harness contributes a directory containing its source code, scores, and execution traces (such as prompts, tool calls, model outputs, and state updates). The filesystem is typically far larger than the proposer's context window, so the proposer queries it through terminal tools such as `grep` and `cat` rather than ingesting it as a single prompt. At each iteration, the proposer first inspects prior code, scores, and execution traces, then reasons about likely failure modes before generating a new harness.

Meta-Harness maintains a population $\mathcal{H}$ and a Pareto frontier over evaluated harnesses, but imposes no parent-selection rule: the proposer is free to inspect *any* prior harness and its execution trace when proposing new ones. We run evolution for a fixed number of iterations and perform a final test-set evaluation on the Pareto frontier. This simplicity is deliberate: by leaving diagnosis and edit decisions to the proposer rather than hard-coding search heuristics, Meta-Harness can improve automatically as coding agents become more capable. The proposer never sees test-set results; its only feedback comes from the **search set**, the subset of task instances used to evaluate candidate harnesses during search and generate the feedback signal for improvement, and from execution traces logged during those search runs.

Advantages of code-space search. Harness optimization occurs in code space, where small changes to retrieval, memory, or prompt-construction logic can affect behavior many steps later, making local search heuristics poorly matched to the problem. By inspecting execution

[1] Based on earlier exploration, we think this workflow only became practical recently, following major improvements in coding-agent capabilities around early 2026.

Algorithm 1 Meta-Harness outer loop over harnesses

Input: tasks $\mathcal{X}$, LLM M, proposer P, iterations N
Initialize: population $\mathcal{H}$ ▷ Initial set of valid harnesses
Initialize: filesystem $\mathcal{D} \leftarrow \emptyset$ ▷ stores code, scores, traces
for $H \in \mathcal{H}$ **do**
  $E_H \leftarrow \text{Evaluate}(H, M, \mathcal{X})$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, E_H)\}$
for $t = 1 \ldots N$ **do**
  Proposer P queries filesystem $\mathcal{D}$ ▷ inspects prior harnesses and scores
  Proposer P proposes k new harnesses $\{H_1, \ldots, H_k\}$
  for H in $\{H_1, \ldots, H_k\}$ **do**
    if H passes interface validation **then**
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, \textsc{Evaluate}(H, M, \mathcal{X}))\}$
return Pareto frontier of harnesses stored in $\mathcal{D}$

traces, the proposer can often infer *why* a harness failed and which earlier design choices likely contributed to the failure, not just *that* it failed, as illustrated by the search trajectories in Appendices A and A.2. There, we see that the proposer reads broadly across prior code and logs, then uses those traces to identify confounded edits, isolate likely causal changes, and shift toward safer modifications after repeated regressions. The proposer can therefore modify the harness at the level of algorithmic structure, ranging from changes to retrieval, memory, or prompt-construction logic to full program rewrites, rather than filling in templates or applying predefined mutation operators. In practice, it often starts from a strong prior harness, but this is an emergent strategy rather than a hard-coded rule. Although the search space is large, representing harnesses as programs provides a natural regularization bias: coding models tend to propose coherent algorithms rather than brittle, hard-coded solutions, which biases the search toward reusable context-management procedures. This action space is closely aligned with the read–write–execute workflows on which frontier coding assistants are trained.

Practical implementation. In our experiments, each harness is a single-file Python program that modifies task-specific prompting, retrieval, memory, and orchestration logic. In our experiments, the proposer P is Claude Code [4] with `Opus-4.6`. The proposer is guided by a minimal domain-specific skill that describes where to write new harnesses, how to inspect previous harnesses and their execution traces, and what files it can and cannot modify. The base model M varies by domain and is always frozen; see Section 4 for details. In our experiments, a typical run evaluates roughly 60 harnesses over 20 iterations. We provide additional tips for implementing Meta-Harness in a new domain in Appendix D.

4 Experiments

We evaluate Meta-Harness on three task domains: online text classification, math reasoning, and agentic coding. In each domain, we compare harnesses discovered by our search against domain-appropriate baselines using the standard evaluation metric. Please refer to each subsection for the precise experimental setup.

We compare against two main classes of methods. **(1) Human-designed strategies**: these are hand-crafted harnesses for each domain, representing the current state of the art in context construction. We describe these baselines in the corresponding subsections. **(2) Program-search methods:** these methods search over candidate harnesses using feedback and reward signals, but are designed for smaller-scale settings than harness engineering.

4.1 Online Text Classification

We follow the online text classification setup of Zhang et al. [59]; Ye et al. [52]: an LLM receives labeled examples one at a time, updates its memory, and is evaluated on a held-out test set. We use `GPT-OSS-120B` as the LLM text classifier, and consider the problem of

Harness	Datasets			Avg.	
	USPTO	S2D	Law	Acc	Ctx ↓
Zero-Shot	12.0	63.2	7.0	27.4	0
Few-Shot (8)	14.0	67.9	21.0	34.3	2.0
Few-Shot (32)	13.0	72.2	21.0	35.4	7.9
Few-Shot (all)	15.0	78.3	29.0	40.8	12.3
MCE [52]†	14.0	83.0	23.0	40.0	28.5
ACE [59]†	**16.0**	77.8	29.0	40.9	50.8
`Meta-Harness`	14.0	**86.8**	**45.0**	**48.6**	11.4

Table 2: Test-set metrics for all harnesses on the three datasets. Ctx denotes additional input tokens in context (thousands). †: implementation from Ye et al. [52]. ↓: lower is better. **Meta-Harness improves online text classification accuracy while using a smaller input context.**

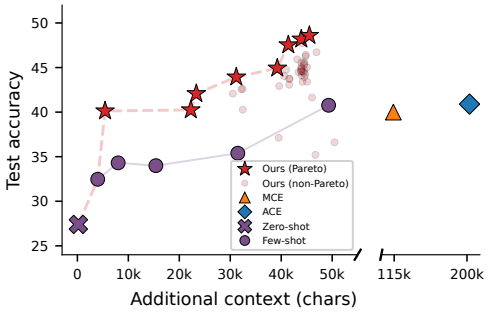

Figure 3: Pareto frontier of accuracy vs. context tokens on online text classification. **Meta-Harness achieves a stronger accuracy-context Pareto frontier than all comparison methods.**

designing a harness for text classification. We use three datasets, chosen for difficulty and domain diversity: **LawBench** (Law) [16] predicts criminal charges from case descriptions (215 classes); **Symptom2Disease** (S2D) [19] predicts diseases from symptom descriptions (22 classes); and **USPTO-50k** [41] predicts precursor reactants from product molecules (180 classes). We initialize the search population $\mathcal{H}$ from the main baseline harnesses in this setting: zero-shot, few-shot, ACE, and MCE. We ran 20 evolution iterations with two candidates per iteration, producing 40 candidate harnesses.

Comparison vs text optimizers. We compare Meta-Harness against representative methods for optimizing text. For a fair comparison, we use the same proposer configuration (`Opus-4.6` with max reasoning), select candidates solely based on search-set performance, and hold out the test sets until the final evaluation. Since evaluation is the main computational bottleneck, we give each method the same budget of proposal harness evaluations. We consider the following points of comparison:

- **Best-of-N**: independent samples from the seed with no search structure; a compute-matched control for whether search matters at all.
- **OpenEvolve** [43]: evolutionary search over programs with LLM mutation.
- **TTT-Discover** [55]: we use only the text-optimization component of their method, i.e., proposal selection via the PUCT reuse rule.

In this setting, Meta-Harness matches the best prior text optimizers (OpenEvolve, TTT-Discover) in $0.1\times$ **the evaluations**, and its final accuracy surpasses theirs by more than 10 points (Figure 1 and Table 4). We attribute this speedup to the intentional design choices that impose minimum necessary structure on the outer loop (Section 3). In particular, Meta-Harness preserves *full experience history using a filesystem* and allows the proposer to inspect anything necessary, whereas both OpenEvolve and TTT-Discover operate with more structured and substantially more limited proposer inputs than full filesystem access. We note that online text classification is the smallest-context setting we study (Table 1), so if structure-heavy text optimizers already lag here, their limitations may only grow in harder regimes.

Meta-Harness is 10× Faster and Converges to a Better Harness

In this setting, Meta-Harness matches the best prior text optimizers (OpenEvolve, TTT-Discover) with 10× fewer full evaluations, and its final accuracy surpasses theirs by more than 10 points.

To isolate which parts of the proposer interface matter most, we compare three conditions in online text classification: a scores-only condition, a scores-plus-summary condition in which the proposer receives LLM-generated summaries but no raw traces, and the full `Meta-Harness` interface with access to execution traces (Table 3). The results show a large gap in favor of the full interface: scores-only reaches 34.6 median and 41.3 best accuracy, while scores-plus-summary reaches 34.9 median and 38.7 best. By contrast, `Meta-Harness`

Method	Scores	Code	Summ.	Traces	Median ↑	Best Acc ↑	> ZS
Scores Only	✓	✓	×	×	34.6	41.3	26
Scores + Summary	✓	✓	✓	×	34.9	38.7	23
`Meta-Harness` (full)	✓	✓	-	✓	**50.0**	**56.7**	39

Table 3: Ablation of the information available to the proposer in online text classification. > ZS: number of runs whose accuracy exceeded the zero-shot baseline. The full Meta-Harness interface substantially outperforms scores-only and scores-plus-summary ablations. **Access to raw execution traces is the key ingredient for enabling harness search.**

reaches 50.0 median and 56.7 best accuracy, and even its median candidate outperforms the best candidate found under either ablation. We interpret this as evidence that full access to execution traces is the most important component of the interface: summaries do not recover the missing signal, and may even hurt by compressing away diagnostically useful details.

Comparison vs state-of-the-art harnesses. Our primary points of comparison are hand-designed harnesses for this problem setting: Agentic Context Engineering (ACE, Zhang et al. [59]), which uses reflective memory curation to build context over time, and Meta Context Engineering (MCE, Ye et al. [52]), which maintains and evolves a library of natural-language skills for context construction. As additional baselines, we evaluate zero-shot prompting and few-shot prompting with $N \in \{4, 8, 16, 32, \text{all}\}$ examples. Results in Table 2 show that Meta-Harness improves substantially over prior hand-designed harnesses. The selected `Meta-Harness`[2] reaches 48.6% accuracy, outperforming ACE by 7.7 points and MCE by 8.6 points. These gains do not come from using more context: `Meta-Harness` uses only 11.4K context tokens, versus 50.8K for ACE and 28.5K for MCE.

Method	Median	Best
GEPA [1]	32.6	40.2
Best-of-N	34.0	44.2
OpenEvolve [43]	39.1	43.3
TTT-Discover [55]	34.1	45.6
`Meta-Harness`	**50.0**	**56.7**

Table 4: Text classification accuracies of the harnesses proposed by different text optimizers (search set). **Meta-Harness is substantially more effective at harness optimization.**

Accuracy–Context Tradeoffs. Because Meta-Harness performs free-form optimization over harness code, we can express a joint preference for both accuracy and context cost rather than committing to a single scalar objective in advance. Given only the current metrics and the desired trade-off, the proposer is able to discover harnesses across a broad range of the frontier, yielding a smooth accuracy–context Pareto curve in Figure 3. This allows us to trade additional context for higher test accuracy in a controlled way, rather than committing to a single hand-designed operating point.

Out-of-distribution (OOD) task evaluation. We evaluate whether the discovered harness generalizes to entirely new datasets unseen during search. We consider nine diverse datasets, and describe them in detail in Appendix C.1. The selected `Meta-Harness` system achieves the best average accuracy (73.1%), outperforming ACE (70.2%) and all few-shot baselines (Table 5). Notably, we observe that naively adding more few-shot examples beyond 32 hurts performance in 7/9 tasks. Meta-Harness shows the highest performance on 6/9 datasets, suggesting that the discovered harness captures generally effective strategies for text classification rather than overfitting to the specific datasets used during search.

4.2 Harnesses for Retrieval-Augmented Reasoning

We study a somewhat non-standard setup for olympiad math solving: augmenting the model with the ability to retrieve examples from a large corpus. There is a good reason to expect retrieval to help mathematical reasoning in principle, because solutions often share reusable proof patterns, so previous reasoning traces contain information that a model may

[2]We slightly overload terminology for brevity: in the tables, `Meta-Harness` denotes the best discovered harness, whereas elsewhere it refers to the entire harness search procedure.

Harness	SciC	FiNER	Amz5	FPB	GoEmo	Bank77	News	SciT	TwHate	Avg Acc	Ctx ↓
Zero-shot	32.7	56.0	52.7	90.0	42.0	80.7	84.7	89.3	75.3	67.0	-
Few-shot (8)	34.0	63.0	54.0	90.0	44.0	82.7	84.7	**91.3**	76.7	68.9	2.2
Few-shot (32)	38.7	62.0	53.3	90.7	43.3	**86.0**	85.3	90.7	76.7	69.6	5.2
Few-shot (all)	35.3	61.0	50.0	93.3	42.7	80.7	84.0	90.0	76.7	68.2	7.4
ACE [59]	40.7	**74.0**	48.0	**96.7**	44.0	83.3	86.0	90.7	68.7	70.2	11.7
`Meta-Harness`	**53.3**	67.0	**60.0**	94.0	**46.0**	82.7	**86.7**	**91.3**	**77.3**	**73.1**	7.3

Table 5: OOD text classification dataset evaluation. We report test accuracy for each dataset and the average additional context tokens across all nine datasets. **`Meta-Harness` outperforms the next best method by 2.9 points on these 9 previously unseen tasks.**

Method	`GPT-5.4n`	`GPT-5.4m`	`Gem-3.1FL`	`Gem-3F`	`GPT-20B`	Avg.
No Retriever	23.0	28.8	28.6	42.6	47.6	34.1
Dense Retrieval (k=1)	27.1 (+4.1)	24.5 (-4.3)	31.3 (+2.7)	42.3 (-0.3)	46.9 (-0.7)	34.4 (+0.3)
Dense Retrieval (k=5)	31.1 (+8.1)	28.3 (-0.5)	37.1 (+8.5)	47.2 (+4.6)	46.7 (-0.9)	38.1 (+4.0)
Random Few-shot	23.1 (+0.1)	24.5 (-4.3)	31.0 (+2.4)	40.4 (-2.2)	41.8 (-5.8)	32.2 (-1.9)
BM25 Retrieval	30.2 (+7.2)	29.2 (+0.4)	32.8 (+4.2)	46.6 (+4.0)	48.9 (+1.3)	37.5 (+3.4)
`Meta-Harness`	31.7 (+8.7)	30.4 (+1.6)	34.9 (+6.3)	46.3 (+3.7)	50.6 (+3.0)	**38.8** (+4.7)

Table 6: Retrieval-augmented math problem solving on 200 IMO-level math problems. We show pass@1 averaged over three samples per problem, with absolute improvement over the baseline in parentheses. **The discovered Meta-Harness retrieval strategy improves reasoning on these IMO-level problems across all five held-out models, with a 4.7-point average gain over no retriever.**

be able to exploit at inference time. Yet retrieval has not become a standard ingredient in this setting, and prior work suggests that it has been much less successful on reasoning-intensive math benchmarks than in more fact-grounded domains [42; 49; 6]. The difficulty is that naive retrieval rarely surfaces the right traces in the right form. This suggests that success depends less on adding retrieval per se than on discovering the right retrieval policy. Rather than hand-designing that policy, we give Meta-Harness a hard set of olympiad problems and allow the retrieval behavior itself to emerge from search.

The retrieval corpus contains ≥500,000 solved problems from eight open-source datasets. We carefully deduplicated and decontaminated it against both evaluation benchmarks and the search set, confirmed that held-out problems have no exact prefix matches under our string-based filter, and manually inspected top BM25 retrievals for held-out examples (appendix C.2). We use Meta-Harness to optimize a harness for 40 iterations over a 250-problem search set of Olympiad-difficulty math problems (OlympiadBench + Omni-MATH hard), producing 109 candidate retrieval harnesses. We initialize the search population $\mathcal{H}$ from the main baseline harnesses in this setting: zero-shot, few-shot, and ACE. We select a single harness based on search-set performance using `GPT-OSS-20B` (Appendix B.2). We evaluate this harness on 200 previously unseen IMO-level problems drawn from IMO-AnswerBench, IMO-ProofBench, and ArXivMath [30; 6]. In addition to `GPT-OSS-20B`, we evaluate the same retrieval harness on four models not seen during search: `GPT-5.4-nano`, `GPT-5.4-mini`, `Gemini-3.1-Flash-Lite`, and `Gemini-3-Flash`. We follow the standard evaluation protocol of prior work [30] and report accuracy averaged over three samples per problem.

Results. Table 6 compares the discovered harness against no retrieval, dense retrieval using the separate embedding model `text-embedding-3-small`, random few-shot prompting, and BM25 retrieval. In contrast, Meta-Harness operates entirely in code space on top of the same BM25-based lexical retrieval stack as the sparse baseline, rather than introducing an additional dense encoder. The discovered retrieval harness outperforms the no-retrieval baseline across all five held-out models, with an average gain of **4.7 points**. It also matches or exceeds the strongest fixed baselines on average, outperforming BM25 retrieval by 1.3 points overall, while avoiding the regressions observed with dense retrieval and random few-shot prompting across several models.

Meta-Harness Improves Reasoning on IMO-Level Math Problems

In retrieval-augmented math reasoning, a single discovered retrieval harness transfers across five held-out models, improving accuracy by 4.7 points on average over no retrieval and yielding the strongest overall average among the compared methods.

4.3 Evaluating Agentic Coding Harnesses on TerminalBench-2

TerminalBench-2 [33] evaluates LLM agents on 89 challenging tasks that require long-horizon, fully autonomous execution under complex dependencies, and substantial domain knowledge. Prior work has shown that the choice agent harness has a large effect on performance on this benchmark. We initialize search from two strong open baselines, Terminus 2 [33] and Terminus-KIRA [25]. For this experiment, we perform search and final evaluation on the same 89-task benchmark. We use this benchmark as a *discovery problem* [54] in which the goal is to discover a harness configuration that improves performance on a hard, publicly contested benchmark. This is standard practice: public writeups already describe repeated benchmark-specific harness iteration on TerminalBench itself [18; 34; 25], and the benchmark is small and expensive enough that introducing a separate split would materially weaken the search signal. We additionally check for overfitting by manual inspection and regex-based audits for task-specific string leakage into evolved harnesses. We note that although the resulting harness is specialized to the TerminalBench-2 regime, autonomous completion of difficult long-horizon tasks from a single instruction is a core capability, and the benchmark consists of many tasks that frontier models and heavily engineered harnesses struggle with.

Results. We report results on the full benchmark in Table 7, evaluated on two base models: `Claude Opus 4.6` and `Claude Haiku 4.5`. On `Opus 4.6`, Meta-Harness discovers a harness achieving 76.4% pass rate, surpassing the hand-engineered Terminus-KIRA (74.7%) and ranking #2 among all `Opus 4.6` agents on the TerminalBench-2 leaderboard. The only higher-scoring `Opus 4.6` agent is ForgeCode (81.8%); however, we were unable to reproduce their reported result from the publicly available code alone, suggesting their leaderboard scores depend on components beyond the published repository. On the weaker `Haiku 4.5` model, the improvement is larger: Meta-Harness achieves 37.6%, outperforming the next-best reported agent (Goose, 35.5%) by 2.1 points. TerminalBench-2 is an actively contested benchmark with multiple teams directly optimizing for it, so the fact that an automatic search method can achieve benefits at this frontier is encouraging for long-horizon text-optimization loops.

Harness	Auto	Pass (%)
`Claude Opus 4.6`		
Claude Code	×	58.0
Terminus 2	×	62.9
Mux	×	66.5
Droid	×	69.9
TongAgents	×	71.9
MAYA-V2	×	72.1
Terminus-KIRA	×	74.7
Capy	×	75.3
ForgeCode	×	81.8
`Meta-Harness`	✓	**76.4**
`Claude Haiku 4.5`		
OpenHands	×	13.9
Claude Code	×	27.5
Terminus 2	×	28.3
Mini-SWE-Agent	×	29.8
Terminus-KIRA	×	33.7
Goose	×	35.5
`Meta-Harness`	✓	**37.6**

Table 7: Pass rate on TerminalBench-2. Results or others are from the official leaderboard. **Meta-Harness ranks #2 among all `Opus-4.6` agents and #1 among all `Haiku-4.5` agents on this competitive task.**

Qualitative behavior of the proposer. The harness search trajectory helps explain *why* Meta-Harness achieves these gains; we provide a detailed summary in Appendix A. In early iterations, the proposer combined plausible structural fixes with prompt-template edits and observed that both candidates regressed. It then explicitly hypothesized that the regressions were confounded by the shared prompt intervention, isolated the structural changes from the prompt rewrite, and ultimately pivoted toward a safer additive modification that became the best candidate in the run. This provides qualitative evidence that **filesystem access enables the proposer to inspect prior experience in enough detail to form causal hypotheses and revise the harness accordingly.**

Meta-Harness Surpasses Hand-Engineered Agents on TerminalBench-2

On TerminalBench-2, Meta-Harness automatically discovers harnesses that surpass Terminus-KIRA on Opus 4.6 and rank #1 among all Haiku 4.5 agents.

5 Discussion

Beyond outperforming existing harnesses, Meta-Harness has several practical advantages. Discovered harnesses generalize to out-of-distribution classification datasets (Table 5) and to unseen base models in the math setting (Table 6). A search run completes in a few hours of wall-clock time, yet produces readable, transferable strategies that can be reused across models, including future, stronger ones. Overfitting in code space is also more inspectable: brittle if-chains or hard-coded class mappings are visible on inspection in a way that weight-space overfitting is not. More broadly, our results suggest that the main advantage of Meta-Harness is not just search over code, but search with *selective access to prior diagnostic experience*. The proposer is not limited to scalar rewards or fixed summaries; it can inspect raw code, execution traces, and prior failures, then use that information to form and test hypotheses about what to change. The qualitative search trajectories in Appendix A.2 illustrate this behavior directly.

Our findings reflect a recurring pattern in machine learning [45]: once a search space becomes accessible, stronger general-purpose agents can outperform hand-engineered solutions. A natural next step for future work is to co-evolve the harness and the model weights, letting the strategy shape what the model learns and vice versa. While we evaluate on three diverse domains, our experiments demonstrate that harness search can work with one particularly strong coding-agent proposer (Claude Code); a broader study of how the effect varies across proposer agents remains for future work.